NutriPredict

E1 : Base de Donnees et Ingestion Data

Candidat : Valentin PERIES

Certification : RNCP37827 - Developpeur en Intelligence Artificielle

Projet : NutriPredict - Prediction d'impact nutritionnel par IA

Date : Juin 2026

C1 C2 C3 C4 C5

E1 - Base de Donnees et Ingestion Data

1. Contexte et objectifs

NutriPredict est une application de prediction d'impact nutritionnel qui exploite les donnees publiques d'OpenFoodFacts pour alimenter un modele de Machine Learning. L'enjeu de ce bloc de competences est de demontrer la capacite a collecter, nettoyer, modeliser et deployer une base de donnees relationnelle conforme aux standards de securite, notamment pour les donnees de sante.

2. Acquisition et nettoyage des donnees (C1, C2)

2.1 Source : OpenFoodFacts

Les donnees brutes proviennent du dump CSV public d'OpenFoodFacts (~3 millions de produits). Le pipeline d'ingestion a ete construit avec les outils suivants :

- **DuckDB** : moteur SQL analytique embarque, utilise pour filtrer les produits pertinents (categorie biscuits) directement en memoire sans charger l'integralite du dump.
- **Pandas** : nettoyage fin des colonnes nutritionnelles (gestion des valeurs manquantes, detection d'anomalies, normalisation des unites).
- **Parquet** : format de stockage colonnaire compresse pour le dataset d'entrainement (4 000 echantillons, 6 colonnes).

2.2 Pipeline de nettoyage

Le nettoyage suit un processus en 4 etapes :

1. **Filtrage** : selection des produits de la categorie "biscuits" avec donnees nutritionnelles completes (energie, glucides, lipides, proteines).
2. **Suppression des doublons** : dedoublonnage par code-barres EAN-13.
3. **Traitement des valeurs aberrantes** : suppression des lignes ou les calories depassent 900 kcal/100g (seuil physique impossible).
4. **Normalisation** : conversion de toutes les valeurs en float64, arrondi a 2 decimales.

> **Resultat :** Le dataset final contient 4 000 echantillons et 6 colonnes (user_bmr, biscuit_calories_100g, portion_g, weight_delta_grams, etc.), stockes en format Parquet (200 Ko compresse).

3. Modelisation Merise (C3)

La base de donnees de NutriPredict a ete modelisee selon la methode Merise, en trois niveaux progressifs :

3.1 Modele Conceptuel de Donnees (MCD)

Le MCD identifie deux entites principales :

- **User** : represente un utilisateur de l'application (email, mot de passe hache).
- **HealthMetrics** : represente une mesure biometrique horodatee (taille chiffree, poids chiffre, metabolisme de base calcule).

La cardinalite est 1:N -- un utilisateur possede plusieurs mesures de sante.

3.2 Modele Logique de Donnees (MLD)

Le MLD traduit le MCD en tables relationnelles :

Table	Colonnes	Contraintes
`user`	id (UUID PK), email (UNIQUE), hashed_password, created_at	Index sur email
`health_metrics`	id (UUID PK), user_id (FK), height_encrypted, weight_encrypted, basal_metabolic_rate, recorded_at	FK avec CASCADE, server_default sur recorded_at

3.3 Modele Physique (MPD) - Implementation SQLAlchemy

Le MPD est directement implemente via l'ORM SQLAlchemy 2.0 avec le style declaratif `Mapped[]`. Les modeles sont definis dans `backend/app/models/`.

```
class HealthMetrics(Base):
    __tablename__ = "health_metrics"
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True,
default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("user.id",
ondelete="CASCADE"))
    height_encrypted: Mapped[str] = mapped_column(String, nullable=False)
```

```
    weight_encrypted: Mapped[str] = mapped_column(String, nullable=False)
    basal_metabolic_rate: Mapped[Optional[float]] = mapped_column(Float,
nullable=True)
    recorded_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now())
```

4. Deploiement PostgreSQL (C4)

La base de donnees est deployee via Docker Compose avec l'image officielle `postgres:16-alpine`. La configuration inclut :

- Un healthcheck automatique (`pg_isready`) qui conditionne le demarrage de l'API.
- Les credentials sont injectes via des variables d'environnement (fichier `.env` non versionne).
- Les tables sont creees automatiquement au demarrage de l'application FastAPI via SQLAlchemy (`Base.metadata.create_all`).

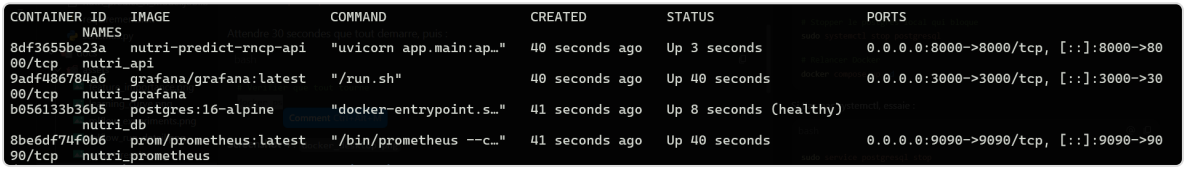


Figure 1 - Conteneurs Docker en execution (PostgreSQL, API, Prometheus, Grafana)

5. API Data - Endpoints FastAPI (C5)

L'API REST exposee par FastAPI permet de manipuler les donnees de la base. Quatre endpoints sont disponibles :

Methode	Endpoint	Description
GET	`/api/v1/products/{barcode}`	Recherche un produit OpenFoodFacts par code-barres
POST	`/api/v1/health-metrics/`	Enregistre une mesure biometrique (donnees chiffrees AES-256)
GET	`/api/v1/health-metrics/latest`	Recupere la derniere mesure de sante
POST	`/api/v1/predict`	Lance l'inference du modele RandomForest

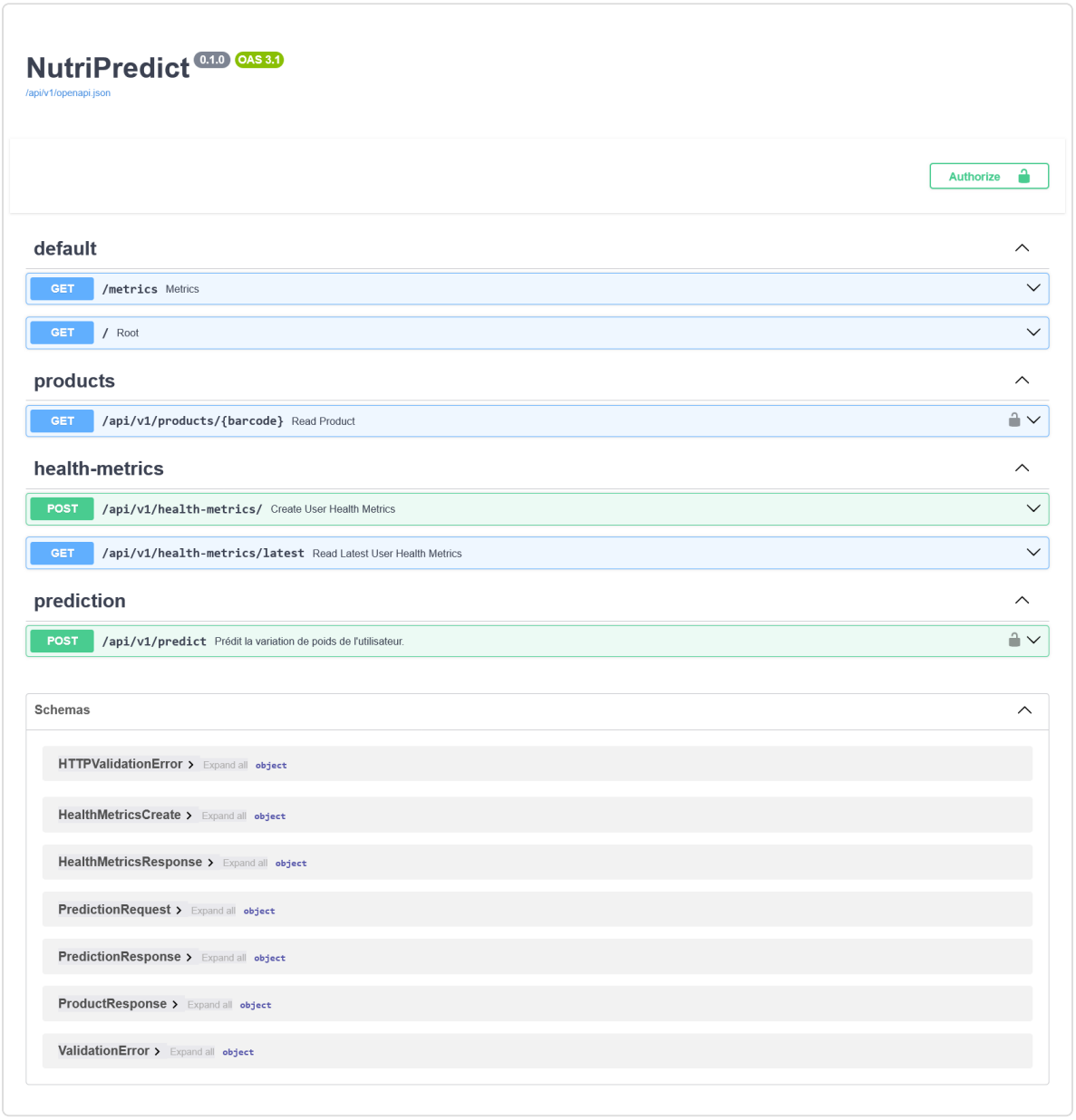


Figure 2 - Documentation interactive Swagger (OpenAPI 3.1)

5.1 Securite des donnees de sante

Les donnees biometriques (taille, poids) sont chiffrees avant insertion en base via **AES-256-GCM** (module `cryptography`). Le processus :

1. Generation d'un nonce aleatoire de 12 octets (`os.urandom(12)`).
2. Chiffrement des donnees avec AESGCM incluant un tag d'authentification.
3. Concatenation `[NONCE(12)][CIPHERTEXT+TAG]`, encodage Base64 URL-safe.
4. Stockage du blob chiffre dans PostgreSQL (colonne `String`).

La cle de chiffrement de 256 bits est stockee exclusivement dans les variables d'environnement, jamais dans le code source.